BUS RESERVATION SYSTEM

Database Management System | B.Tech CSE Capstone Project

TEAM MEMBERS

SHYAM NATH PATRO

AVINASH MK

PRIYANSHU SINGH

ARIJIT DEB

SHASWAT DWIVEDI

PROJECT OVERVIEW

A comprehensive, database-driven web application designed to automate and streamline the management and ticketing operations of a bus transport network — replacing manual ledgers with a robust digital infrastructure.

CRUD

Passengers

Fleet Mgmt

Buses

Tracking

Conductors

Generation

Tickets

Tech Stack

MySQL | Node.js | HTML/CSS/JS | REST API

PROBLEM STATEMENT

01 Data Inconsistency

Passenger info, tickets, and schedules stored in disparate, unlinked files leading to conflicting data.

02 Redundancy

Same passenger or conductor details entered multiple times across different logs, wasting storage and effort.

03 No Real-time Tracking

Administrators cannot instantly ascertain total bookings, available buses, or conductor assignments.

04 Poor Reporting

Generating daily manifests (e.g., matching passengers to bus routes) is time-consuming and error-prone.

PROJECT OBJECTIVES

Relational Modeling

Architect a structurally sound ER model encompassing Passengers, Buses, Conductors, and Tickets.

Data Normalization

Normalize the schema up to 3NF to eliminate insertion, update, and deletion anomalies.

Interactive UI

Build a responsive, dark-themed frontend dashboard for seamless CRUD operations.

Backend API

Develop a Node.js server to securely process API requests and execute SQL queries dynamically.

Advanced SQL

Implement multi-table Joins, Views, and Aggregate functions for unified operational reports.

SYSTEM ARCHITECTURE

FRONTEND

HTML5 | CSS3 | JavaScript

BACKEND

Node.js | HTTP Module | REST API

DATABASE

MySQL | Normalized Schema | Views

KEY FEATURES

- SPA with Fetch API
- Parameterized Queries
- ON DELETE CASCADE
- Dark Theme UI
- PaaS Deployment

ER MODEL & SCHEMA

PASSENGERS

p_id (PK)
first_name, last_name
gender, age
phone, email, address

BUSES

b_id (PK)
b_number (UNIQUE)
bus_name, bus_type
total_seats, source, destination
departure_time, arrival_time

CONDUCTORS

c_id (PK)
first_name, last_name
phone, email
experience_years

TICKETS

t_id (PK)
ticket_number (UNIQUE)
booking_date, journey_date
seat_number, fare

Junction Tables: passenger_ticket | passenger_bus | conductor_bus | conductor_ticket

NORMALIZATION

1NF

First Normal Form

Every attribute must be atomic — no repeating groups.

Name split into first_name & last_name. Phone and email in distinct columns. No multi-valued ticket arrays.

2NF

Second Normal Form

All non-key attributes fully depend on the entire primary key.

Single-column surrogate keys (b_id, c_id) inherently satisfy 2NF. Junction tables contain only key mappings.

3NF

Third Normal Form

No transitive dependencies — non-key attrs depend only on the PK.

Passenger age lives in passengers, not tickets. Tables connected via junction tables eliminate transitive dependencies.

DATABASE DESIGN

passengers

Attribute	Type	Constraint
p_id	INT	PK, AUTO_INC
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
gender	ENUM	NOT NULL
age	INT	NOT NULL
phone	VARCHAR(15)	UNIQUE
email	VARCHAR(100)	UNIQUE

buses

Attribute	Type	Constraint
b_id	INT	PK, AUTO_INC
b_number	VARCHAR(20)	UNIQUE
bus_name	VARCHAR(100)	NOT NULL
bus_type	ENUM	NOT NULL
total_seats	INT	NOT NULL
source / dest	VARCHAR(100)	NOT NULL
dep / arr_time	TIME	NOT NULL

Referential Integrity: Foreign Keys with ON DELETE CASCADE ensure no orphaned records. Junction tables (passenger_ticket, passenger_bus, conductor_bus, conductor_ticket) resolve many-to-many relationships.

>_ SQL: VIEWS & JOINS

VIEW: passenger_ticket_details

Merges passengers + passenger_ticket (junction) + tickets via INNER JOIN to show which passenger holds which ticket — displaying passenger_name, ticket_number, seat_number, and fare.

```
SELECT p.p_id, CONCAT(first_name,' ',last_name) AS passenger_name,
  ticket_number, seat_number, fare
FROM passengers p
JOIN passenger_ticket pt ON p.p_id = pt.p_id
JOIN tickets t ON pt.t_id = t.t_id
```

VIEW: passenger_bus_details

Links passengers to bus operational details via INNER JOIN through passenger_bus junction — showing passenger_name, bus_number, bus_name, bus_type, route, and schedule.

```
SELECT p.p_id, CONCAT(first_name,' ',last_name) AS passenger_name,
  b_number, bus_name, bus_type, source, destination,
  departure_time, arrival_time
FROM passengers p
JOIN passenger_bus pb ON p.p_id = pb.p_id
JOIN buses b ON pb.b_number = b.b_number
```

FRONTEND & BACKEND

FRONTEND

HTML5
Semantic structure for dashboard, nav bars, and data-entry forms

CSS3
Dark theme with CSS variables (--bg: #0D0D0D, --primary: #32CD32). Grid & Flexbox for responsive layouts.

JavaScript
DOM manipulation, section toggling, form capture, async Fetch API calls to backend.

BACKEND

Node.js (Native HTTP)
Server on port 3000. Manual URL parsing, CORS headers, JSON body processing, route handling — no Express.

MySQL2 Driver
Connection pool via db.js. Environment variables (dotenv) for secure credential management.

Security
Parameterized queries (Prepared Statements) to prevent SQL Injection. ACID compliance enforced.

RESULTS & FEATURES

01

Admin Dashboard

Real-time aggregated metrics: passenger count, bus fleet, conductors, and tickets — fetched dynamically from DB.

02

Passenger Module

Full CRUD with data entry form, search functionality, and tabulated record view.

03

Bus Management

Route planning, capacity tracking, bus type classification (AC, Non-AC, Sleeper, Semi-Sleeper).

04

Conductor Tracking

Profile management with contact info and experience years for scheduling.

05

Ticket Generation

Unique ticket codes, seat assignment, booking/journey dates, and fare recording.

06

SQL Reports

Predefined Views generating flattened, human-readable passenger-ticket and passenger-bus reports.

CONCLUSION & FUTURE SCOPE

CONCLUSION

The project successfully demonstrates end-to-end development of a robust database application — from ER modeling to 3NF normalization, from a lightweight Node.js API to a responsive dark-themed UI. It eliminates manual redundancies and enables administrators to execute complex logistical workflows with speed and accuracy.

FUTURE SCOPE

User Authentication	JWT-based role-based access control (Admin vs. Operator)
Payment Gateway	Real-time financial transactions and billing history
Seat Selection UI	Interactive graphical bus layout for seat picking
Auto Notifications	Email/SMS API for automated ticket confirmations

THANK YOU